CV-lab2 实验报告

杨子涵

2025 年 10 月 27 日

目录

1 实现思路 **2**

1.1 gradient_x, gradient_y 2

1.2 harris_response 2

1.3 corner_selection 3

1.4 histogram_of_gradients 4

1.5 compute_homography 4

1.6 align_pair 4

1.7 stitch_blend 4

1.8 generate_panorama 5

1.9 others 6

2 运行结果 **8**

1 实现思路

1.1 gradient_x, gradient_y

原本是用自己手写的 `for` 循环，之后改进为 NumPy 的向量化操作，速度有明显优化（见图 1）。

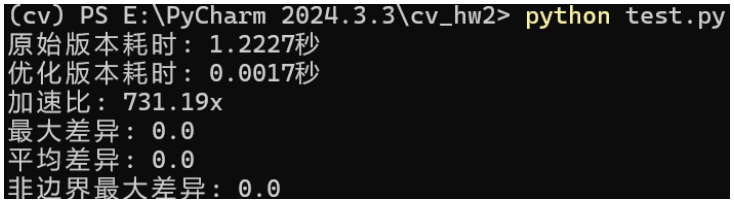


图 1: 优化前后的速度对比

1.2 harris_response

最初使用上一个作业中手写的 `gaussian_blur_kernel_2d` 与 `cross_correlation_2d`。询问助教后得知"not required" 是" 不是必须"，而非" 不可使用"，于是改用库中自带的函数来加速实现（同样也替换了之前手写的 `arctan2` 实现）。

在高斯核的选择上，我将 σ 设为 win_size/6。原理是：希望高斯核的有效范围覆盖整个窗口，考虑 99.7% 的能量在 $\pm 3\sigma$ 内，因此令 $6\sigma = \text{win_size}$。

我尝试了在计算梯度之前先做一次高斯模糊（见图 2a,2b）。虽然两幅图视觉上差别不大，但在打印图像拼接时匹配上的特征点数占总特征点数的比例（代码中 `feature_matching` 函数里的 `rate`）显示：若不加高斯平滑，比例明显下降（见图 3）。原因推测是：不平滑时，噪声导致角点检测出现许多虚假特征，从而降低了匹配成功率；而最终拼接效果变化不大，可能是因为匹配时采取随机采样子集并取匹配数最多的采样结果，这使得无效匹配对最终变换影响有限。

(a) 计算梯度前不进行高斯模糊

(b) 计算梯度前进行高斯模糊

```
(cv) PS E:\PyCharm 2024.3.3\cv_hw2> python homework2.py
rate:0.0593607305936073
rate:0.09581881533101046
rate:0.12680115273775217
rate:0.11544715447154472
(cv) PS E:\PyCharm 2024.3.3\cv_hw2> python homework2.py
rate:0.03241296518607443
rate:0.06149193548387097
rate:0.06502636203866433
rate:0.07777777777777778
```

图 3: 是否高斯平滑对匹配率的影响. 上面是计算梯度前先进行高斯平滑，下面是计算梯度前不进行高斯平滑

1.3 corner_selection

该函数为朴素实现。下图（图 4）展示了在 (min_dist) 分别为 3,5,7,9,11,13 时，匹配上的特征占所有特征的比例。总体趋势是 (min_dist) 越大，比例越高：因为纹理丰富区域会产生许多相似特征，可能导致错配，保留响应最大的点可以提高特征独特性。但 (min_dist) 不能过大：当 (min_dist=13) 时比例反而下降，可能是因为一个小范围内多个有用且独特的特征只保留了一个。

```
(cv) PS E:\PyCharm 2024.3.3\cv_hw2> python homework2.py
rate:0.0593607305936073
rate:0.09581881533101046
rate:0.12680115273775217
rate:0.11544715447154472
(cv) PS E:\PyCharm 2024.3.3\cv_hw2> python homework2.py
rate:0.07931034482758621
rate:0.10263157894736842
rate:0.1422319474835886
rate:0.1291139240506329
(cv) PS E:\PyCharm 2024.3.3\cv_hw2> python homework2.py
rate:0.08866995073891626
rate:0.09523809523809523
rate:0.134375
rate:0.15073529411764705
(cv) PS E:\PyCharm 2024.3.3\cv_hw2> python homework2.py
rate:0.08280254777070063
rate:0.115
rate:0.14410480349344978
rate:0.15897435897435896
(cv) PS E:\PyCharm 2024.3.3\cv_hw2> python homework2.py
rate:0.09917355371900827
rate:0.08609271523178808
rate:0.1488095238095238
rate:0.1360544217687075
(cv) PS E:\PyCharm 2024.3.3\cv_hw2> python homework2.py
rate:0.0660377358490566
rate:0.07563025210084033
rate:0.14516129032258066
rate:0.13675213675213677
```

图 4: min_dist 不同取值下的匹配比例

1.4 histogram_of_gradients

按照课件流程朴素实现。其中把 360 度分成的区域数作为超参数，实验中发现对结果影响不大，最终取 8。

1.5 compute_homography

按照课件上的流程实现，利用了奇异值分解，原理就是奇异值分解后，V 的最后一列（注意 `np.linalg.svd` 的三个返回值是 U, Σ, V^T）对应的是 A^TA 的最小特征向量。至于为什么要选择最小的特征向量，课件上的解释如下：

- 理论上需要 4 个非共线点（单应性矩阵有 8 个自由度），实际应用中我们可以找到更多对应点对。相当于要求解 $A_{(2n\times 9)}h_{(9)} = 0_{(2n)}$
- 转化为在约束 $\|h\|_2 = 1$ 条件下最小化能量函数 $E = \|Ah\|^2$
- 使用拉格朗日乘数法：$E = \|Ah\|^2 + \lambda(\|h\|^2 - 1) = h^TA^TAh + \lambda h^Th - \lambda$
- 对 E 求导并令导数为零：$A^TAh = \lambda h$,$E = \lambda$

因此 h 是 A^TA 的特征向量，对应最小的特征值 λ。但是我不会矩阵求导。其实也可以利用线性代数的知识解决：我们要求解在 $\|h\|_2 = 1$ 的条件下 $\|Ah\|$ 的最小值。由于 A^TA 是一个实对称矩阵，它一定可以正交相似于一个对角矩阵 D，即 $A^TA = P^TDP$，其中 D 的对角线上是 A^TA 的特征值。因此：

$$\|Ah\|^2 = (Ah)^T(Ah) = h^TA^TAh = h^TP^TDPh = (Ph)^TD(Ph)$$

令 $Ph = (y_1, \ldots, y_n)^T$，则 $y_1^2 + \cdots + y_n^2 = (Ph)^T(Ph) = h^TP^TPh = 1$ 于是：

$$\|Ah\|^2 = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2$$

在约束条件 $y_1^2 + y_2^2 + \cdots + y_n^2 = 1$ 下，该式的最小值等于最小的特征值。

1.6 align_pair

朴素地实现。

1.7 stitch_blend

初始的 `alpha_blending` 会在拼接交界处产生黑缝（见图 5a），原因是交界处权重从 0 到 1 存在突变。为此尝试了另一种权重方法：对画布上任意公共区域的点，使用该点到两个图像边界的最近距离来作为权重，以保证权重从边界到内部平滑变化。（代码实现见 utils.py 内的 my_stitch_blend_with_distance 函数）我尝试了两种不同权重计算方法：

- **代码 1（线性权重）：**

```
dist1 = cv2.distanceTransform(mask1, cv2.DIST_L2, 5)
dist2 = cv2.distanceTransform(mask2, cv2.DIST_L2, 5)
weight1 = dist1 / (dist1 + dist2 + 1e-6)
```

- **代码 2（指数权重）：**

```
alpha = 0.1
weight1 = np.exp(-alpha * dist2) /
          (np.exp(-alpha * dist1) +
           np.exp(-alpha * dist2) + 1e-8)
```

理论上 代码 2 更平滑，但最终效果差异不大。

此外尝试了基于高斯金字塔与拉普拉斯金字塔的图像融合（代码实现见 utils.py 内的 my_stitch_blend_with_pyramid 函数）。原理如下：先对两张图像分别构建高斯金字塔，再由高斯金字塔推导拉普拉斯金字塔。从拉普拉斯金字塔顶层开始，逐层将两幅图对应层做加权平均并与当前输出合成，再上采样到下一层尺寸，重复直到底层。拉普拉斯金字塔保存高频信息，高斯金字塔顶层保存低频信息——按层融合可以分离并分别处理高低频细节。但直接使用 01 掩码会在边界处产生突变，导致仍有明显缝隙（见图 5b）。后来将金字塔融合时使用基于距离的平滑权重（代码实现见 utils.py 内的 my_stitch_blend_with_pyramid_and_distance 函数），得到图 5c，交界处缝隙基本消失，但每张图片的四个角落仍有未完全融合的现象。

(a) alpha_blending 导致黑色缝隙

(b) 金字塔融合（使用 01 掩码）导致透明缝隙

(c) 金字塔融合（使用距离加权）

1.8 generate_panorama

助教提供的原始代码的思路是以中间图像为基准向两侧扩展：每次把一张图片拼接进来并用当前拼接结果与下一张匹配再拼接，但这样会导致已放到画布上的图片被不断拉伸，特征变形，从而影响后续匹配，甚至出现内存占用过高（见图 6）或匹配失败（assert `rate >= 0.03` 不通过）。

我的改进为：先计算好所有相邻两张图片的变换矩阵，再把每张图片直接放到目标画布上的最终位置（每张图的最终变换矩阵为若干相邻变换矩阵的连乘）。从匹配成功率的统计来看，该改进显著降低了失败概率，并且避免了内存占用过高的问题。

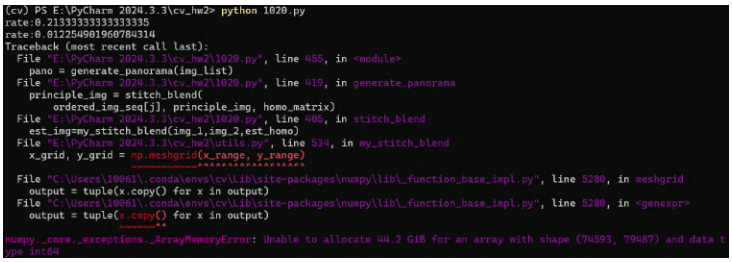


图 6: 创建的画布尺寸为 (74593, 79487)，需要 44.2G 的内存（fig9）

不过，由于得分的标准是能生成 5 张图拼接成的全景图，所以我采用了原始代码。

1.9 others

我本来让 AI 帮我写了一份代码（代码实现见 utils.py 内的 generate_comprehensive_report 函数），遍历若干组超参数组合，选择最好的超参数,（最好的标准是拼接上的图片数量最多）, 输出类似于图 7。

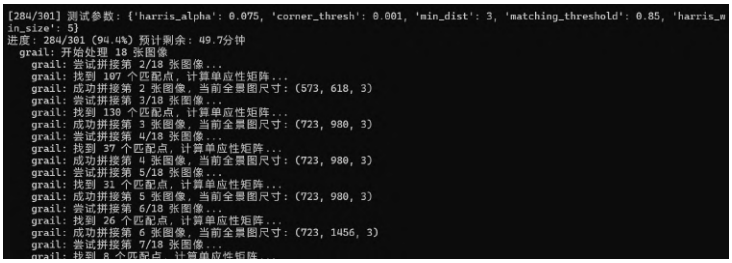


图 7: 超参数调优的输出结果示例

但是遍历完所有结果之后发现最优解仅仅是因为 thresh 很大，所以才能将很多图片拼接在一起，效果类似于《星际穿越》中的五维空间（见图 8）。后来我又分别固定 thresh=0.7,0.75,0.8 遍历其他超参数，但是仍然发现，能够拼接上的图片数量最多不等于效果最好（可能有重影/过度拉伸）。总之，让 AI 帮忙写自动遍历超参数的代码这条路失败了，因为程序的运行结果好坏需要人看到生成的全景图才能判断。

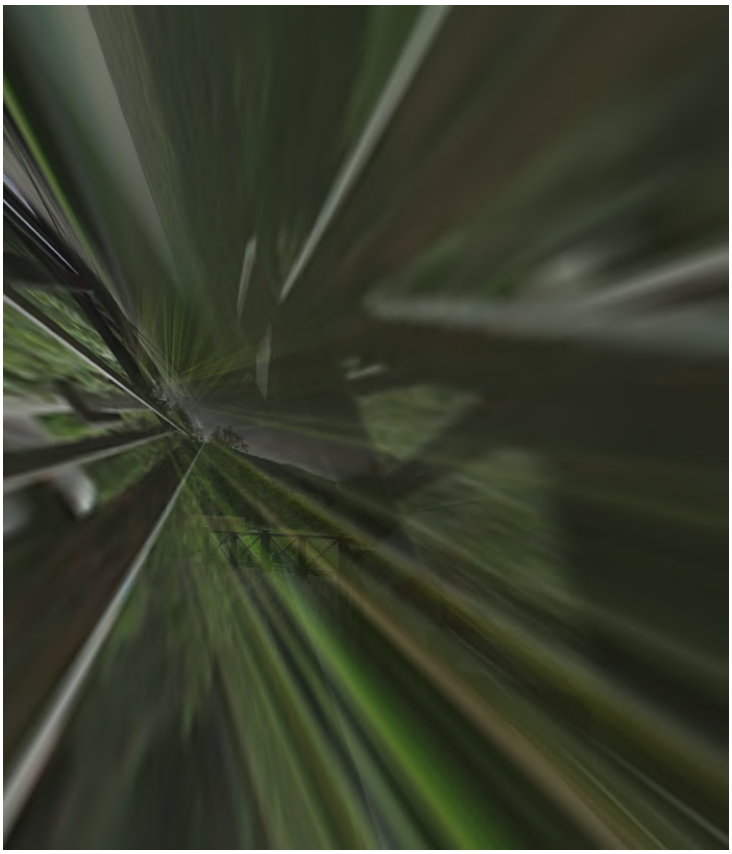


图 8: 调大 thresh 将所有图片拼接在一起的效果

2 运行结果

在助教给出的图片中测试，结果如下。生成的 7 张图片所用的超参数都是 alpha=0.035,win_size=9,corner_thresh=0.009,min_dist=7,thresh=0.8,num_bins=4。固定种子为 2400012732（我的学号）。生成的四张全景图分别是第 0-4,4-8,0-4,0-4 张图片的拼接结果。直接运行 python homework2.py 可复现出这 7 张图片的结果。

(a) stitch1

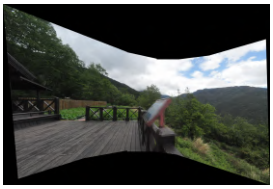
(b) stitch2

(c) stitch3

(a) panorama1

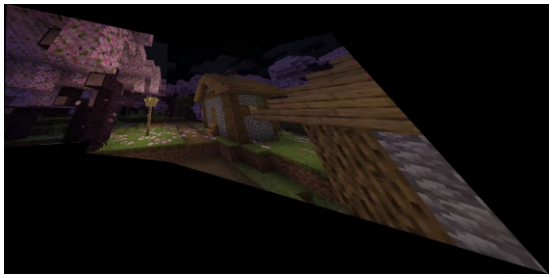
(b) panorama2

(c) panorama3

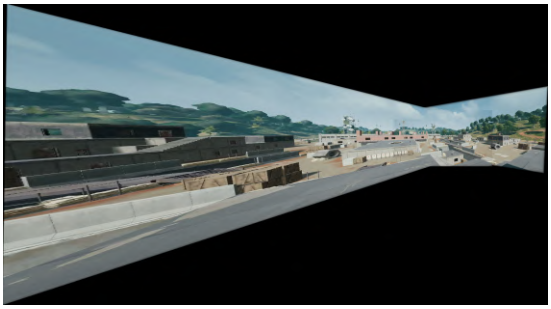


(d) panorama4

我自己在 Minecraft 和 PUBG 中分别收集了数据并且尝试拼接全景图，但是拼接效果不佳（见图 11a 11b）。我猜测是游戏内视角的问题导致本身显示的图像就经过了一定的拉伸。我在不同的视角下测试了生成的结果，但是效果差距不大。



(a) Minecraft 拼接结果



(b) PUBG 拼接结果

我也在 Habitat-Sim 仿真器中收集截图测试了图像拼接的效果。相比于以上两款游戏，视角范围导致的图像拉伸不太明显。测试结果是，如果平行移动相机，则可以拼出全景图（见图 12）。如果相机转动的话，就拼不出全景图。（相机转动时采集的图片见图 13）

图 12: Habitat-Sim 中得到的全景图

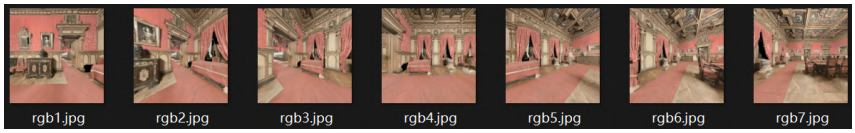


图 13: 相机转动时采集的单张图片

总的来说，得出的结论在收集图片的时候，相机的朝向最好不变，并且垂直于相机朝向移动相机。如果相机视角转动，那么拼出来的全景图就会产生拉伸，很可能会超过画布的大小。